6. Nuevo Input System (acciones, eventos y lectura de inputs)

Unity introdujo el **Nuevo Input System** como alternativa al sistema tradicional basado en Input.GetAxis y Input.GetKey. Este sistema es más flexible, permite crear mapas de control complejos y ofrece soporte nativo para gamepads, teclado, móvil y más.

6.1. ¿Qué es una InputAction?

Una **InputAction** representa una acción abstracta que el jugador puede realizar, como:

- Moverse
- Saltar
- Disparar
- Interactuar

No importa si esa acción se dispara desde teclado, mando o pantalla táctil: la InputAction se encarga de unificarlo.

6.2. InputActionReference

En los scripts se usa:

```
[SerializeField] InputActionReference mover, saltar, atacar;
```

Un **InputActionReference** es un objeto que referencia una acción creada en el archivo de InputActions del proyecto.
Esto permite arrastrar desde el inspector la acción correspondiente y reutilizarla en distintos scripts.



6.3. Ciclo de vida: OnEnable / OnDisable

Al trabajar con eventos del Input System, es obligatorio suscribirse y desuscribirse correctamente.

Ejemplo del script **Jugador**:

```
void OnEnable()
{
    atacar.action.started += Atacar;
}
void OnDisable()
{
    atacar.action.started -= Atacar;
}
```

Y en el script **Humano**:

```
void OnEnable()
{
    atacar.action.started += Atacar;
    saltar.action.started += Saltar;
}
void OnDisable()
{
    atacar.action.started -= Atacar;
    saltar.action.started -= Saltar;
}
```

Esto evita **errores, duplicación de eventos** y problemas cuando el objeto se desactiva/activa.

6.4. Eventos del Input System

Cada InputAction dispara varios eventos:

- started: cuando el input comienza
- performed: cuando la acción se ha ejecutado
- canceled: cuando se deja de pulsar

Ejemplo en el script **Jugador**:

```
private void Atacar(InputAction.CallbackContext obj)
{
    TripleDisparo();
}
```

En el script **Humano**, al atacar se activa un trigger del Animator:

```
private void Atacar(InputAction.CallbackContext obj)
{
    animator.SetTrigger("atacar");
}
```

6.5. Leer inputs: ReadValue()

Para inputs continuos (como movimiento), el Nuevo Input System permite leer valores directamente.

Ejemplo en el script **Jugador**:

```
Vector2 direccion = mover.action.ReadValue<Vector2>();
```

Ejemplo en **Humano**, donde el movimiento es horizontal:

```
float direccion = mover.action.ReadValue<float>();
```

El tipo genérico (Vector2, float, etc.) depende del tipo de acción configurado en el InputAction.

6.6. Modos típicos de InputAction

"Value"

Devuelve un valor continuo.
Ejemplo: movimiento Vector2.

"Button"

Sirve para detectar pulsaciones.
Ejemplo: saltar o disparar.

"Pass Through"

Permite combinar inputs sin bloqueo entre dispositivos.

6.7. Ejemplo completo de flujo de entrada

Caso: Movimiento del script Jugador

1. El jugador mueve joystick o teclas (WASD o flechas).
2. La InputAction "mover" registra los cambios.
3. El script ejecuta cada frame:

```
Vector2 direccion = mover.action.ReadValue<Vector2>();

transform.position += velocidad * (Vector3)direccion *
Time.deltaTime;
```

Caso: Salto en el script Humano

1. El jugador pulsa el botón configurado como “saltar”.
2. Se dispara el evento .started.
3. El script ejecuta:

```
if (enSuelo)

    rb.linearVelocityY = fuerzaSalto;
```

6.8. Uso combinado del Input System con Animator + físicas

El script **Humano** integra InputActions con:

- Animator (para atacar, moverse, saltar)
- Rigidbody2D (salto y movimiento)
- HUD (actualizar barra de munición)

El Input System actúa como **capa inicial** que activa todas las demás acciones del personaje.

